Java Interview Preparation

Part 6: System Design, SQL Practice, Debugging & Behavioral Prep

Section 1: System Design Fundamentals (Q1–Q15)

Q1: What is horizontal vs vertical scaling, and when do you pick one over the other?

ANSWER

Vertical Scaling (Scale Up): Adding more CPU/RAM/disk to a single server. Simple, no code changes, but has a hard ceiling and creates a single point of failure.

Horizontal Scaling (Scale Out): Adding more machines behind a load balancer. Needs stateless services and a shared data layer, but scales near-indefinitely and improves fault tolerance.

At 3 YOE, the expected answer: start vertical for simplicity until it is clearly insufficient, then move to horizontal once traffic/data outgrows a single box, and the app is stateless enough to support it.

Q2: What does a Load Balancer do, and what are common algorithms?

ANSWER

Distributes incoming traffic across multiple backend instances to avoid overloading any one server and to provide failover if an instance dies.

Round Robin: Requests distributed sequentially across servers.

Least Connections: Routes to the server with fewest active connections.

IP Hash: Routes based on client IP, useful for session stickiness.

Layer 4 (Transport) vs Layer 7 (Application): L4 balances based on IP/port; L7 can route based on URL path, headers, cookies.

Q3: When and why would you introduce a caching layer (e.g., Redis)?

ANSWER

Introduce caching when read traffic is high relative to writes, and the same data is requested repeatedly (e.g., product catalog, user session, config).

Cache-Aside (Lazy Loading): App checks cache first; on miss, reads DB and populates cache. Most common pattern.

Write-Through: Every write goes to cache and DB together, keeping cache always fresh at the cost of write latency.

TTL and eviction policy (LRU/LFU) matter: always design for staleness and set appropriate expiry to avoid serving stale data indefinitely.

Q4: What is Database Sharding and when do you need it?

ANSWER

Sharding splits a large dataset across multiple database instances (shards), usually by a shard key (e.g., user ID, region).

Needed when a single DB instance can no longer handle the write throughput or data volume, even after indexing and read replicas.

Trade-off: cross-shard joins and transactions become hard; shard key choice is critical and hard to change later. At 3 YOE, you're expected to explain the concept and trade-off, not necessarily implement it.

Q5: What is the difference between a Message Queue and a Pub/Sub system (Kafka vs RabbitMQ style)?

ANSWER

Message Queue (point-to-point): Each message consumed by exactly one consumer (e.g., RabbitMQ classic queue). Good for task distribution/work queues.

Pub/Sub (Kafka-style): Messages published to a topic; multiple independent consumer groups each receive a full copy of the stream. Good for event broadcasting, decoupling services, and replayability since Kafka retains messages for a configurable period.

Use a queue for "do this job once"; use pub/sub for "notify many systems this happened."

Q6: Explain the CAP Theorem in practical terms.

ANSWER

In a distributed system, during a network partition (P, which is unavoidable), you must choose between Consistency (every read gets the latest write) and Availability (every request gets a response, possibly stale).

CP systems (e.g., traditional RDBMS in strict mode, Zookeeper): prioritize correctness, may reject requests during partition.

AP systems (e.g., Cassandra, DynamoDB default mode): prioritize uptime, may serve stale data temporarily.

Practical answer in interviews: name a real system you'd pick for a given scenario (e.g., inventory counts need consistency; social media feed can tolerate staleness).

Q7: How would you design a URL Shortener (like bit.ly)?

ANSWER

Core requirement: given a long URL, generate a short unique code; given the short code, redirect to the original URL.

Approach: Use a counter-based ID (auto-increment, base62-encoded) or a hash of the URL with collision handling. Store mapping in a key-value store (fast lookups) with a relational DB as source of truth if analytics are needed.

Scaling: Cache hot URLs in Redis since redirects are read-heavy. Use a CDN/edge layer for redirect latency. Shard by short-code prefix if write volume is extreme.

Extras to mention: expiry/TTL for links, custom aliases, click analytics via async event logging (don't block the redirect path).

Q8: How would you design a basic Rate Limiter?

ANSWER

Purpose: prevent a client from exceeding N requests per time window to protect backend resources.

Token Bucket: A bucket holds tokens refilled at a fixed rate; each request consumes a token; reject if empty. Allows bursts up to bucket size.

Sliding Window Log/Counter: Track request timestamps (or counts per sub-window) within the last N seconds; more accurate than fixed windows but more memory.

Implementation: Use Redis with atomic INCR + EXPIRE (or a Lua script for atomicity) so the limiter works correctly across multiple app instances.

Q9: How would you design a basic Notification System (push/email/SMS)?

ANSWER

Core flow: An event occurs (e.g., order placed) -> event published to a queue/topic -> Notification Service consumes it -> determines channel preference (push/email/SMS) -> calls the relevant provider (FCM/SendGrid/Twilio) -> logs delivery status.

Key design points: Decouple notification sending from the core business transaction using async messaging, so a slow email provider never blocks order placement. Add retry with backoff for transient provider failures. Use a template system so content isn't hardcoded per notification type.

Scale consideration: batch low-priority notifications; prioritize transactional ones (OTP, payment confirmation) over marketing ones.

Q10: How would you design a simplified E-Commerce Order Flow?

ANSWER

Core services: Catalog Service, Cart Service, Order Service, Payment Service, Inventory Service — kept loosely coupled.

Flow: Place order -> reserve inventory (avoid overselling) -> process payment -> on success, confirm order and decrement inventory permanently; on failure, release the reservation.

Key design point: Use a distributed transaction pattern (Saga) instead of a two-phase commit across services — e.g., if payment fails after inventory reservation, publish a compensating event to release stock.

Idempotency matters: payment and order-confirmation endpoints must be idempotent (using an idempotency key) since retries are common in distributed systems.

Q11: What is the difference between a Monolith and Microservices, and when would you NOT recommend microservices?

ANSWER

Monolith: single deployable unit, simpler to develop/test/deploy initially, but scaling and team-ownership boundaries get harder as the codebase grows.

Microservices: independently deployable services, better for scaling specific parts and independent team ownership, at the cost of operational complexity (service discovery, distributed tracing, network latency, data consistency across services).

When NOT to use microservices: small team, early-stage product, unclear domain boundaries — premature microservices create huge overhead without the scaling benefit. This is a common trap question at 3 YOE to test judgment over buzzword knowledge.

Q12: How do you handle Database Read Scaling before resorting to sharding?

ANSWER

Read Replicas: Asynchronously replicate the primary DB to one or more read-only replicas; route read-heavy queries (reports, listings) to replicas, writes to primary.

Caching layer: Offload repeated reads to Redis/Memcached before touching the DB at all.

Query optimization and indexing: Often the real fix is a missing index or an N+1 query, not infrastructure scaling. Always rule this out before reaching for replicas/sharding.

Connection pooling: Ensure the app isn't exhausting DB connections under load (e.g., HikariCP tuning in a Spring Boot app).

Q13: What is Eventual Consistency, and where is it acceptable?

ANSWER

A model where, after a write, all reads will eventually (not immediately) reflect that write, since propagation across replicas/caches takes time.

Acceptable: social media likes/counts, search index updates, analytics dashboards — brief staleness doesn't harm correctness.

Not acceptable: bank account balance after a withdrawal, inventory count at checkout, authentication/session state — these need strong consistency.

Interviewers often probe whether you can tell these apart for a given feature rather than reciting the definition.

Q14: How would you design a simple real-time Chat/Messaging feature at a basic level?

ANSWER

Use WebSockets (or Server-Sent Events for one-directional updates) instead of polling, so the server can push messages to connected clients immediately.

Store messages in a DB (per conversation), and use a message broker (Kafka/Redis Pub-Sub) to fan out messages across multiple WebSocket server instances, since a message sender and recipient may be connected to different app servers.

Scale consideration: use a "presence service" to track which server a user is connected to, so you can route the message correctly without broadcasting to every instance.

Q15: What questions should you ask BEFORE designing any system in an interview?

ANSWER

Scale: How many users/requests per second? Read-heavy or write-heavy?

Consistency requirements: Can this tolerate eventual consistency, or does it need strong consistency?

Latency requirements: Is this a real-time feature or can it be slightly delayed (async)?

Data size and growth: How much data, and how fast is it growing?

This is one of the most commonly missed steps at 3 YOE — jumping straight to a solution without clarifying requirements signals junior-level thinking to the interviewer. Always start here before drawing any boxes.

Section 2: SQL Practice Queries (Q16–Q30)

Q16: Write a query to find the second-highest salary from an Employees table.

ANSWER

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Alternative using window function (handles ties differently):
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 2;
```

Q17: Write a query to find duplicate rows in a table based on email column.

ANSWER

```
SELECT email, COUNT(*) AS occurrences
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Q18: Write a query to find employees who earn more than their manager.

ANSWER

```
SELECT e.name AS employee_name, e.salary, m.name AS manager_name, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Q19: Write a query using a window function to rank employees by salary within each department.

ANSWER

```
SELECT
    name,
    department,
    salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;
```

Q20: Explain and write an example of ROW_NUMBER() vs RANK() vs DENSE_RANK().

ANSWER

ROW_NUMBER(): Always assigns unique sequential numbers, even for ties (1,2,3,4...).

RANK(): Ties get the same rank, but the next rank skips numbers (1,1,3,4...).

DENSE_RANK(): Ties get the same rank, but the next rank does NOT skip (1,1,2,3...).

```
SELECT name, salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
    RANK() OVER (ORDER BY salary DESC) AS rank_val,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_val
FROM employees;
```

Q21: Write a query to find the Nth highest salary generically using LIMIT/OFFSET.

ANSWER

```
-- Find the 3rd highest salary (MySQL/Postgres style)
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Q22: Write a query to get the running total of sales per day.

ANSWER

```
SELECT
    sale_date,
    daily_total,
    SUM(daily_total) OVER (ORDER BY sale_date) AS running_total
FROM (
    SELECT sale_date, SUM(amount) AS daily_total
    FROM sales
    GROUP BY sale_date
) daily;
```

Q23: Write a query to find customers who have placed orders in every month of 2025.

ANSWER

```
SELECT customer_id
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2025
GROUP BY customer_id
HAVING COUNT(DISTINCT EXTRACT(MONTH FROM order_date)) = 12;
```

Q24: Explain the difference between INNER JOIN, LEFT JOIN, and a self-join, with an example.

ANSWER

INNER JOIN: Returns only rows with matches in both tables.

LEFT JOIN: Returns all rows from the left table, with NULLs where there's no match on the right.

Self-Join: A table joined with itself, typically used for hierarchical data (e.g., employee-manager).

```
-- Find all employees, including those with no assigned manager
SELECT e.name, m.name AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Q25: Write a query to find products that have never been ordered.

ANSWER

```
SELECT p.product_id, p.name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;

-- Alternative using NOT EXISTS (often better performance on large tables):
SELECT p.product_id, p.name
FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);
```

Q26: Write a query to pivot rows into columns (e.g., total sales per quarter as columns).

ANSWER

```
SELECT
    product_id,
    SUM(CASE WHEN quarter = 'Q1' THEN amount ELSE 0 END) AS q1_sales,
    SUM(CASE WHEN quarter = 'Q2' THEN amount ELSE 0 END) AS q2_sales,
    SUM(CASE WHEN quarter = 'Q3' THEN amount ELSE 0 END) AS q3_sales,
    SUM(CASE WHEN quarter = 'Q4' THEN amount ELSE 0 END) AS q4_sales
FROM quarterly_sales
GROUP BY product_id;
```

Q27: What is the difference between WHERE and HAVING, with an example showing why order matters?

ANSWER

WHERE filters rows BEFORE grouping/aggregation; HAVING filters groups AFTER aggregation. You cannot use an aggregate function like COUNT() in WHERE.

```
-- Find departments with more than 5 employees earning over 50000
SELECT department, COUNT(*) AS headcount
FROM employees
WHERE salary > 50000      -- filters individual rows first
GROUP BY department
HAVING COUNT(*) > 5;      -- filters the grouped result
```

Q28: Write a query to detect gaps in a sequence of IDs (missing order numbers).

ANSWER

```
SELECT (order_id + 1) AS gap_start
FROM orders o1
WHERE NOT EXISTS (
    SELECT 1 FROM orders o2 WHERE o2.order_id = o1.order_id + 1
)
AND order_id < (SELECT MAX(order_id) FROM orders);
```

Q29: Explain how an index speeds up a query, and write an example of when a query WON'T use an index.

ANSWER

An index (typically B-Tree) allows the DB to locate rows without scanning the entire table, similar to a book's index — turning an O(N) scan into roughly O(log N) lookup.

A query often WON'T use an index when: a function is applied to the indexed column (e.g., WHERE LOWER(email) = 'x'), a leading wildcard is used (LIKE '%term'), or there's an implicit type mismatch. Example:

```
-- Won't use an index on 'created_at' efficiently:
SELECT * FROM orders WHERE YEAR(created_at) = 2025;

-- Better (sargable — index-friendly):
SELECT * FROM orders WHERE created_at >= '2025-01-01' AND created_at < '2026-01-01';
```

Q30: Write a query to find the top 3 highest-paid employees per department using window functions.

ANSWER

```
SELECT name, department, salary
FROM (
    SELECT name, department, salary,
        DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk <= 3;
```

Section 3: Production Debugging & Scenario-Based Questions (Q31–Q40)

Q31: An API endpoint that used to respond in 200ms is now taking 5 seconds in production. Walk through how you'd diagnose it.

ANSWER

1. Check if it's isolated: Is it one endpoint or all endpoints slow? Check APM/monitoring (e.g., New Relic, Grafana) for latency trends and correlate with a deploy or traffic spike.

2. Check the DB layer first: Run EXPLAIN ANALYZE on the underlying query — a missing index, a new N+1 query introduced in a recent change, or table growth are the most common causes.

3. Check for lock contention: Long-running transactions or missing indexes can cause row/table locks that queue up other requests.

4. Check downstream calls: If the endpoint calls another service/API, check if that dependency slowed down.

5. Check resource saturation: CPU, memory, DB connection pool exhaustion (e.g., HikariCP pool maxed out causing requests to wait for a connection).

Expected interview behavior: narrate a structured, layered diagnosis (app -> DB -> infra -> dependencies) rather than guessing randomly.

Q32: A Spring Boot application is throwing intermittent OutOfMemoryError in production. How do you investigate?

ANSWER

1. Capture a heap dump at the time of the crash (-XX:+HeapDumpOnOutOfMemoryError) and analyze it with Eclipse Memory Analyzer (MAT) to find what's retaining memory.

2. Common causes: unbounded caches/collections (e.g., a static Map growing forever), ThreadLocal values not cleared in a pooled-thread environment, large result sets loaded fully into memory instead of streamed/paginated.

3. Check GC logs to see if it's a genuine leak (old gen keeps growing after full GC) vs. just undersized heap for legitimate load.

4. Fix: bound caches with eviction (Caffeine/Guava), paginate large queries, clear ThreadLocals in a finally block, and tune heap size appropriately — don't just bump -Xmx as the only fix.

Q33: Your service occasionally returns duplicate records when a user double-clicks "Submit" or a client retries a request. How do you prevent this?

ANSWER

Implement idempotency: require the client to send a unique idempotency key with the request (often a UUID generated client-side). The server stores processed keys (with a TTL) and, on a duplicate key, returns the original response instead of processing again.

At the DB level, add a unique constraint on a natural business key (e.g., order reference) so even a race condition can't create duplicates — let the DB be the final safety net, not just application logic.

Q34: You deployed a change and now see a spike in 500 errors, but only for 5% of requests. How do you approach this?

ANSWER

Check if the 5% correlates with something specific: a particular user segment, request payload shape, specific server instance (if only one pod/instance is unhealthy after a bad deploy), or a specific downstream dependency only some requests hit.

Check logs for the actual stack trace behind the 500s — don't guess before reading the exception.

If it's deploy-related and severity is high, the first action is often to roll back, then investigate with logs/APM afterward — restoring service takes priority over root-causing live in production.

Q35: A batch job that processes 1 million records is taking hours and sometimes times out. How would you optimize it?

ANSWER

Batch the DB writes: instead of one INSERT/UPDATE per record, use batch inserts (JDBC batching, Hibernate batch_size) to drastically cut round trips.

Avoid loading everything into memory: stream/paginate from the DB instead of fetching all 1M rows into a List.

Parallelize where safe: split into chunks and process concurrently (ExecutorService or Spring Batch's partitioning) if records are independent.

Remove unnecessary work inside the loop: e.g., re-opening a DB connection per record, or doing a lookup query inside a loop (classic N+1 in batch jobs).

Q36: Two services need to update their own databases as part of one logical business transaction (e.g., Order Service and Inventory Service). How do you keep them consistent without a distributed transaction?

ANSWER

Use the Saga pattern: each service performs its local transaction and publishes an event; the next service reacts to that event. If a later step fails, compensating events undo the earlier steps (e.g., release reserved inventory if payment fails).

Alternative for simpler cases: Outbox pattern — write the business change and an "event to publish" in the same local DB transaction, then a separate process reliably publishes it to the message broker, avoiding the "DB committed but message lost" problem.

Q37: Your application's database CPU usage is constantly near 100% during peak hours. What's your investigation approach?

ANSWER

1. Identify the top expensive queries via the DB's slow query log or performance schema.

2. Check for missing indexes on frequently filtered/joined columns using EXPLAIN.

3. Check for N+1 query patterns from the application layer (common with lazy-loaded JPA/Hibernate associations).

4. Consider read replicas to offload reporting/read-heavy queries from the primary.

5. Check connection pool settings — too many idle-in-transaction connections can also spike load.

Q38: A REST API you own is being called by multiple downstream teams, and you need to change the response structure. How do you do this without breaking existing consumers?

ANSWER

Never break a published contract silently. Options: version the API (/v2/...), add new fields without removing old ones (additive, backward-compatible changes), or use a deprecation period — announce the old field/endpoint as deprecated, give consumers a migration window, monitor usage of the old field until it drops to zero, then remove it.

This tests whether you think about API consumers as a real constraint, not just your own service's convenience — a common theme at 3 YOE where you're expected to own a service others depend on.

Q39: You suspect a memory leak from improper use of ThreadLocal in a Spring Boot app running on a thread pool. Explain the root cause and the fix.

ANSWER

Root cause: In a pooled-thread environment (e.g., Tomcat's request-handling threads), threads are reused across requests. If a ThreadLocal value is set during a request but never removed, it stays attached to that thread and leaks into the next unrelated request that reuses the same thread — both a memory leak and a correctness bug (stale data crossing requests).

Fix: Always call threadLocal.remove() in a finally block once the value is no longer needed, typically in a filter/interceptor that wraps the request lifecycle.

Q40: How would you explain "why did this bug happen and how did you fix it" for a past production issue in a way that lands well in an interview?

ANSWER

Structure it as: (1) Symptom — what was observed (e.g., a specific error, slow endpoint, wrong data). (2) Investigation — the steps you took to isolate the cause, not just the final answer (this is what interviewers actually want to hear). (3) Root cause — the precise technical reason. (4) Fix — what you changed. (5) Prevention — what you added afterward (a test, a monitor/alert, a code review checklist item) so it doesn't recur.

Interviewers weight the investigation process and the prevention step heavily — jumping straight to "I fixed it by changing X" without explaining how you found X, or stopping at the fix without mentioning prevention, is a common way candidates undersell a good story.

Section 4: Behavioral & Project Depth Prep (Q41–Q50)

Q41: What is the STAR format, and how do you use it for behavioral answers?

ANSWER

Situation: Brief context — what was the setting (1-2 sentences, don't over-explain).

Task: What were you specifically responsible for or trying to achieve.

Action: What YOU did, specifically — this is the bulk of the answer. Say "I", not "we", when describing your own contribution, even within a team effort.

Result: The outcome, ideally with a measurable impact (time saved, bug reduced, performance improved), plus what you learned.

Keep each answer to roughly 1.5–2 minutes — too short feels unprepared, too long loses the interviewer.

Q42: "Tell me about a time you disagreed with a technical decision." How should you structure this?

ANSWER

Pick a real disagreement with a reasonable, not hostile, framing. Explain: what the proposed approach was, why you disagreed (your technical reasoning), how you raised it (data, prototype, or a calm conversation — not just complaining), and the outcome — including if you were overruled and how you handled that professionally.

Interviewers are testing communication and collaboration maturity, not whether you were "right." A good answer shows you can disagree constructively and commit to a team decision once made, even if it wasn't yours.

Q43: "Tell me about a time you had to learn something new quickly." What makes this answer strong?

ANSWER

Strong answers show a repeatable learning process, not just "I read the docs": e.g., how you scoped what you actually needed to learn (vs. everything), where you found reliable sources, how you validated your understanding (a small prototype, pairing with someone experienced), and how you applied it under a real deadline.

Weak answers are vague ("I just Googled it and figured it out") with no structure or measurable outcome.

Q44: "Tell me about a mistake you made in production." How honest should you be, and how do you frame it?

ANSWER

Be genuinely honest — interviewers can tell when a "mistake" story is actually a humble-brag ("my biggest flaw is I work too hard"). Pick a real mistake with real consequences, own it without excessive self-flagellation or blaming others, and spend most of the answer on what you did to fix it and what changed afterward (a new test, a new review step, a new monitor).

This question is really testing accountability and growth, not perfection.

Q45: How do you talk about a project you built (e.g., an ERP module) in a way that shows seniority appropriate for 3 YOE?

ANSWER

Don't just describe what the feature does — describe the decisions: why you chose a particular data model, what trade-off you made under a constraint (time, existing architecture, team size), what you'd do differently with hindsight, and how you handled an edge case or bug that came up after launch.

At 3 YOE, interviewers expect you to have opinions about your own past decisions — "it works" is a fresher-level answer; "it works, and here's what I'd change now and why" is a 3 YOE-level answer.

Q46: "Why are you leaving/looking to leave your current role?" How do you answer this without sounding negative?

ANSWER

Frame around what you're moving toward, not what you're escaping: growth, scope, technology, or team structure you're seeking — rather than complaining about your current company, manager, or colleagues, even if those are the real reasons.

Never badmouth a previous employer in an interview — it reads as a risk signal regardless of how justified the complaint is.

Q47: "How do you prioritize when you have multiple tasks with conflicting deadlines?" What's a strong structure for this answer?

ANSWER

Mention a concrete framework you actually use: impact vs. effort, business priority (checking with your lead/PM when unclear), or identifying blocking dependencies (what's blocking others vs. what only blocks you).

Give one specific real example rather than speaking only in generalities — "I use urgency vs. importance" with no example sounds rehearsed and unconvincing.

Q48: How should you answer "Do you have any questions for us?" at the end of an interview?

ANSWER

Always have 2-3 ready. Good categories: team/codebase ("what does the on-call/incident process look like?", "what's the biggest technical debt the team is dealing with right now?"), growth ("what does progression from this role typically look like?"), and culture ("how does the team handle code review/disagreements?").

Avoid asking only about salary/benefits at this stage, and avoid "no questions" — it signals low engagement.

Q49: How do you handle a technical question you genuinely don't know the answer to?

ANSWER

Don't bluff. Say what you DO know that's adjacent, reason out loud toward a plausible answer using first principles, and clearly state what you're unsure about. Interviewers consistently rate "I don't know, but here's how I'd find out / here's my best reasoning" far higher than a confident wrong answer or a long silence.

This matters as much in system design and SQL questions as in pure trivia.

Q50: What's a good way to close out an interview and follow up afterward?

ANSWER

End by briefly reiterating genuine interest in the role and thanking them for their time. If you have a strong, specific reason you want the role (not generic), it's fine to mention it briefly.

Send a short thank-you note/email within 24 hours if you have the interviewer's contact — not mandatory everywhere, but it's a low-cost, positive signal, especially after an onsite/final round.